

Laboratorio 7 - Rendering di una scena contenente diversi oggetti

In questa esercitazione ripartiamo dal programma del laboratorio precedente, estendendolo con le seguenti caratteristiche:

- Refactoring del codice.
- Trackball per il controllo della camera (paradigma World-in-hand). La trackball non ha le limitazioni della camera orbitante usata fino ad ora.
- Uso delle trasformazioni di modellazione per posizionare un oggetto nella scena.
- Composizione di una scena contenente diversi oggetti.
- Navigazione in prima persona dentro la scena, con controllo diretto del mouse (paradigma Camera-in-hand).

Capitolo 0 - Template: refactoring

Il codice finale del laboratorio precedente viene modificato, mantenendo inalterate le sue funzionalità. Le modifiche principali sono le seguenti:

1. I moduli header-only `hotshaders.hh` e `mesh.hh` all'interno della directory `include` vengono modificati, incapsulandone i contenuti dentro il namespace `fcg`. Nel corso del laboratorio verranno introdotti altri moduli ancora, e tutti lavoreranno dentro questo namespace.
2. Introduciamo un nuovo modulo `matrices.hh` contenente le funzioni principali per definire le matrici di base utilizzate nella modellazione: traslazione, rotazioni attorno agli assi principali e scalatura. **Nota:** la libreria `glm` fornisce funzionalità analoghe nelle estensioni che implementano funzionalità legacy da OpenGL 1.0. Preferiamo definire esplicitamente queste funzioni, per due ragioni: perché la sintassi di quelle presenti in libreria non è di immediata comprensione, e per favorire la comprensione di quali siano i blocchi basilari che compongono le trasformazioni e del loro rapporto diretto con la teoria. Tuttavia, una volta capito come funzionano, può essere utile ricorrere direttamente a quelle, specialmente per operazioni più sofisticate, come ad esempio la rotazione attorno a un asse arbitrario, il posizionamento della camera mediante l'operazione `lookAt`, o la definizione della matrice di proiezione.
3. Nel programma, separiamo la classe `CameraLights` del programma precedente in due classi separate `Camera` e `Lights` che gestiscono rispettivamente la camera e le luci. Nell'ambito di queste classi, utilizziamo direttamente le trasformazioni definite nel modulo `matrices.hh` dove necessario (nota: la matrice di proiezione prospettica resta definita direttamente nel codice

della classe **Camera**). La classe **Camera** tiene separati i parametri intrinseci ed estrinseci della telecamera:

- Si assume che il centro della scena sia posizionato a una distanza `od` dall'osservatore lungo l'asse di vista della telecamera.
 - Gli angoli `phi_deg` e `theta_deg` definiscono l'orientamento della telecamera, mentre la rotazione attorno all'asse di vista è fissa e assume un vettore view-up sempre parallelo alla verticale del mondo.
 - La distanza focale `fd` definisce indirettamente l'apertura della visuale: a tale distanza dall'osservatore, la sezione della piramide di vista è un quadrato di lato 2; il valore di `fd` è anche la cotangente della semi-apertura dell'angolo di vista (per ora uguale in orizzontale e verticale, perché si assume un volume di vista a sezione quadrata). Le costanti `normal_fd`, `tele_fd` e `wide_fd` definiscono tre distanze focali corrispondenti ad obiettivi 50mm, 400mm e 24mm di una macchina fotografica full frame (sensore o pellicola 35mm).
4. Nel modulo `mesh.hh` carichiamo una mesh senza ricentrarla e riscalarla esplicitamente: la mesh resta fedele alle coordinate di modellazione che aveva nel file, la funzione di load si limita a calcolare la sua estensione.
 5. Definiamo una nuova classe **MeshGPU** per gestire sia il caricamento della mesh (tramite la classe **Mesh**), sia tutti i dettagli per trasferirla alla GPU. In questa classe, calcoliamo il centro della mesh e la sua estensione. L'idea è che potranno co-esistere numerose istanze di **MeshGPU** caricate contemporaneamente in memoria.
 6. Nella classe **Scene**, utilizziamo il metodo di caricamento della classe **MeshGPU** per caricare la mesh e trasferirla alla GPU: notiamo che i vertici della mesh vengono trasferiti alla GPU con le loro coordinate originali (quindi espresse nel sistema di riferimento locale alla mesh, detto *sistema dell'oggetto*).
 7. Sempre nella classe **Scene** calcoliamo la trasformazione di modellazione (traslazione e scalatura) necessaria a centrare la mesh sull'origine del mondo e scalarla in modo che sia contenuta in un cubo di lato 1. La scalatura è uniforme, ovvero mantiene intatte le proporzioni originali dei dati. Calcoliamo inoltre la trasformazione da applicare alle normali (calcolate dalla classe **Mesh** durante il caricamento) per esprimerle in coordinate del mondo. Questa trasformazione sostituisce la normalizzazione della mesh effettuata in fase di lettura nella versione precedente: ora i vertici della mesh sono memorizzati (e trasferiti alla GPU) con le loro coordinate originali, mentre la trasformazione di modellazione da applicare per riscalarla e centrare la mesh viene memorizzata a parte e poi fornita come uniform al vertex shader in fase di rendering. **Nota:** l'espressione generale della matrice che trasforma le normali è l'inversa trasposta di quella che trasforma la mesh stessa.
 8. Modifichiamo in modo consistente tutti i vertex shader, in modo che

applichino la matrice di modellazione prima di applicare quella `vp`; analogamente, applichiamo la trasformazione di modellazione alle normali. Modifichiamo il programma in modo che queste matrici siano passate come uniform ai vari vertex shader.

9. Aggiungiamo alla callback per l'evento `KeyPressed` della tastiera la gestione del tasto `Escape`. Premere `ESC` farà terminare il programma tramite `exit(0)`.

Capitolo 1 - Trackball

Sostituiamo il controllo camera eliminando il precedente modello di camera orbitante e implementando il modello di trackball basato su quaternioni. La trackball viene implementata nel modulo `trackball.hh` e utilizzata all'interno della classe `Camera` per calcolare la componente di rotazione della trasformazione di vista:

1. La classe `Trackball` usa i membri privati `current_rotation` e `drag_rotation` per memorizzare due quaternioni che ricordano rispettivamente la rotazione applicata *prima* del movimento del mouse e quella accumulata *durante* il drag. La rotazione viene aggiornata accumulando la `drag_rotation` sulla `current_rotation`. **Nota:** un quaternioni rappresenta una rotazione di un certo angolo attorno a un certo asse; la composizione di due rotazioni si calcola come prodotto dei due quaternioni; il risultato viene convertito nella matrice di rotazione equivalente tramite la funzione di libreria `glm::toMat4`.
2. Il calcolo della rotazione viene effettuato dalla funzione membro `move`, che usa al suo interno la funzione `project_on_trackball` per proiettare le coordinate mouse sulla trackball virtuale. La funzione `project_on_trackball` usa un [metodo](#) di proiezione diverso e più sofisticato di quello visto a lezione: fornisce comunque un vettore che dipende dalla posizione del puntatore del mouse rispetto al centro di rotazione; la funzione `move` usa i vettori calcolati sulle posizioni del mouse al frame corrente e al precedente per calcolare sia l'asse sia l'angolo di rotazione (come visto a lezione).
3. La classe `Camera` gestisce le rotazioni della trackball attraverso le tre funzioni `start_rotate`, `rotate` e `stop_rotate`. La rotazione da applicare viene ottenuta dalla funzione `view-projection`, che viene invocata anche in fase di inizializzazione e dalle funzioni che cambiano l'obiettivo `lens_*`.
4. Il rapporto d'aspetto della sezione del volume di vista viene ora calcolata nella classe `Camera`, tramite il metodo `set_window_size`, in modo da rispettare quello della viewport di visualizzazione; tale rapporto viene memorizzato nel membro privato `ar` e utilizzato per costruire la matrice di proiezione. È quindi possibile utilizzare una viewport rettangolare senza introdurre distorsioni di scalatura tra `x` e `y`. **Nota:** da qui in poi, per la trackball si utilizza l'apertura *verticale* dell'angolo di vista, la cui

cotangente è data dal rapporto $1/(fd * ar)$.

Capitolo 2 - Trasformazioni di modellazione

Fino a questo punto, abbiamo applicato le trasformazioni di modellazione a un singolo oggetto per normalizzarlo, centrandolo sull'origine e riscalandolo perché sia contenuto nel cubo di lato 1. Questa prima trasformazione di modellazione, calcolata dalla classe **Scene** dopo il caricamento e memorizzata nella matrice **mesh_mm**, ci consegna un oggetto prototipale che può essere ulteriormente scalato/ruotato/traslato per posizionarlo nella scena, accumulando le ulteriori trasformazioni di modellazione sulla stessa matrice **mesh_mm**.

Con un uso opportuno delle trasformazioni di modellazione, si possono costruire scene di dimensione e complessità arbitraria; potenzialmente, ci si può svincolare dalla necessità di mantenere tutto dentro il cubo di dimensioni $[-1,1]$.

Assumendo di mantenere ancora il centro della scena sull'asse di vista, per una data apertura della piramide di vista (stabilita dai parametri **fd** e **ar**), la porzione di scena inquadrata dipende unicamente dalle dimensioni della scena e dalla distanza **od** del suo centro dall'osservatore: a parità di apertura, maggiore è la distanza, maggiore è la porzione inquadrata (e minore la dimensione percepita degli oggetti nella scena); a parità di distanza, maggiore è l'apertura, maggiore è la porzione inquadrata. I metodi **lens_*** cambiano contemporaneamente la distanza focale e la distanza dell'oggetto dall'osservatore per mantenere approssimativamente il cubo di lato $[-1,1]$ nel volume di vista. Per una scena diversa - più o meno estesa rispetto al cubo normalizzato - il rapporto tra **fd** e **od** dovrebbe essere scalato diversamente.

In questo capitolo, ci limitiamo ad una scena contenente un solo oggetto, che viene modificato e traslato fuori dal centro della scena, ma ancora contenuto nel cubo di lato $[-1,1]$. A titolo di esempio, nel programma **02_modelmatrix.cc**, modifichiamo solo la funzione **draw** della classe **Scene** in modo che l'unico oggetto caricato venga:

- Riscalato schiacciandolo lungo il suo asse z.
- Ruotato di 90 gradi attorno all'asse x.
- Traslato lungo l'asse y.

Notiamo che l'oggetto viene visualizzato verso la periferia della viewport, mentre la rotazione della trackball avviene ancora attorno al centro della vista: agendo sulla trackball, l'oggetto si sposta nello spazio ruotando attorno a tale punto.

Capitolo 3 - Scena contenente diversi oggetti

La classe **Scene** viene modificata ulteriormente per caricare diverse mesh e posizionarle indipendentemente una dall'altra in modo da costruire una scena.

In particolare, il cubo viene istanziato più volte, riscalandolo opportunamente, per costruire:

- Un ambiente (`half_room`) costituito da un pavimento e due pareti.
- Diversi tavoli, ciascuno costituito da un piano e quattro gambe, posizionati ai quattro angoli dell'ambiente.
- Oggetti riscaldati in diversi modi, posizionati sui quattro tavoli.

La classe `Scene` contiene un membro di tipo `GPUMesh` per ogni mesh caricata da file; nel metodo `draw`, utilizza le funzioni di draw dei vari oggetti, assegnando a ciascuno la trasformazione di modellazione necessaria a posizionarlo opportunamente nella scena.

La gerarchia degli oggetti viene gestita *senza usare un grafo di scena*, definendo diverse funzioni di `draw`: da quelle che disegnano le singole mesh, alle `draw_half_room` e `draw_table` che usano diverse mesh per modellare rispettivamente l'ambiente e un tavolo, alla `draw_table_with_objects` che assembla un tavolo con oggetti, alla `draw` principale della classe `Scene`, che usa le precedenti a cascata per assemblare il tutto.

Le varie funzioni `draw_*` prendono tutte un parametro `parent_mm`, che è la matrice di modellazione “genitore” con cui vogliamo disegnare l'oggetto specifico. La funzione `Scene::draw`, che è la radice delle funzioni gerarchiche di disegno, parte da una matrice `root` che al momento è inizializzata all'identità. Se venisse inizializzata diversamente, tutti gli oggetti nella scena erediterebbero a cascata questa trasformazione. Provate a scalare e/o traslare tutta la scena tramite la matrice `root`, o provate a scalare o traslare solo delle parti.

Ad esempio, per la matrice `root`:

- Eventuali traslazioni spostano la scena dal centro della trackball.
- Scalature non uniformi di tutta la scena hanno effetti di distorsione globale immediatamente visibili.
- Una scalatura uniforme di tutta la scena la renderebbe più grande o più piccola del cubo unitario. Se fate una scalatura che la ingrandisce di molto (diciamo di 10 volte) quanti e quali parametri dovete cambiare, e come, nel resto del programma per tornare ad una visualizzazione equivalente a quella precedente? Provate a portare questo esercizio anche nei capitoli successivi (paradigma Camera-in-hand vs paradigma World-in-hand).

Nota: Per scene più complesse, l'uso di un grafo di scena è molto più conveniente ed è raccomandato (leggere le estensioni proposte per maggiori dettagli).

Capitolo 4 - Paradigma Camera-in-hand

In questo capitolo, cambiamo il paradigma di interazione da *World-in-hand* a *Camera-in-hand*, consentendo all'osservatore di muoversi all'interno della scena. I controlli di movimento sono gestiti come segue:

- Il movimento del mouse controlla l'orientamento della telecamera attraverso i due angoli `phi_deg` e `theta_deg`, in modo simile alla camera orbitante, ma ora il centro di rotazione è la camera stessa e non il centro della scena.
- La barra spaziatrice (eventualmente modificata con `ALT` per movimenti all'indietro) controlla la traslazione del punto di vista all'interno della scena: se la barra è premuta, il punto di vista avanza lungo il vettore di vista di un offset proporzionale al tempo trascorso ad ogni frame.

Tutti gli eventi di movimento vengono gestiti all'interno della classe `Camera` dalle funzioni `pan_tilt*` per quanto riguarda la rotazione e dalle funzioni `dolly*` per quanto riguarda la traslazione. Le funzioni `handle` del mouse e della tastiera vengono modificate di conseguenza.

La posizione della camera e i due angoli che ne descrivono l'orientamento vengono utilizzati come in precedenza dalla funzione `view_projection` della classe `Camera` per definire la matrice di vista. Per una miglior comprensione, confrontare questa versione della `view_projection` con quella che implementa la camera orbitante in `00_refactor.cc`. Qual'è il cambiamento fondamentale?

Capitolo 5 - Miglior gestione della rotazione

La gestione del mouse implementata nel capitolo precedente ha alcune limitazioni: in particolare, la rotazione richiede un'azione di drag e si arresta quando il mouse raggiunge i limiti della finestra.

Per un miglior controllo - ad esempio necessario in videogiochi che richiedono questo paradigma di interazione - la rotazione può essere gestita nel modo seguente:

- L'applicazione cattura il mouse e ne nasconde il puntatore. La cattura può essere automatica alla partenza del gioco, e il mouse può venire liberato quando l'utente interagisce tramite tastiera per uscire dal gioco o per mettere in pausa. Nel nostro caso, visto che la nostra applicazione non usa i click del mouse, facciamo sì che un click del tasto sinistro attivi e disattivi la cattura e la visibilità del mouse (un singolo click genera un toggle immediato: non c'è gestione del rilascio del bottone).
- L'obiettivo è che ogni movimento del mouse controlli in maniera diretta il mutamento della direzione di vista nell'applicazione. Il puntatore del mouse viene nascosto in quanto la posizione assoluta del puntatore non ha più significato: ci interessano solo i movimenti relativi del mouse. Anzi, il puntatore del mouse sarebbe un impedimento, in quanto il mouse si bloccherebbe una volta raggiunti i bordi dello schermo. Noi vogliamo che invece i movimenti relativi del mouse continuino ad arrivare "all'infinito" all'applicazione.
- Quando questo tipo di interazione è stato inventato, la tecnica di realizzazione era quella del cosiddetto *mouse warp*: l'applicazione nascondeva il

puntatore, ne prendeva il controllo, lo spostava nel centro della finestra e ad ogni movimento dello stesso calcolava gli offset e lo rimetteva al centro.

- Ad oggi, i maggiori sistemi operativi forniscono tutti dei metodi per accedere al cosiddetto input *raw* del mouse, ovvero non filtrato dal sistema operativo. Così facendo, arrivano in input i dati generati direttamente dalla device, che in ogni momento fornisce proprio dei delta, ovvero lo spostamento relativo del mouse sulla superficie su cui si sposta. Questo offre diversi vantaggi, in particolare vengono saltati: eventuali accelerazioni del mouse mediate dall'OS; e eventuali scalature delle coordinate dovute a schermi ad alta densità di pixel (scalature inevitabili se il nostro input è la posizione assoluta del puntatore del mouse dentro uno schermo o una finestra, posizione che è mediata dal sistema operativo).
- La gestione della lettura dei valori relativi di spostamento *raw* (non mediati dall'OS) differisce per il sistema MacOS rispetto ai sistemi Windows e Linux. Le necessarie opzioni di compilazione per rendere il programma portabile vengono gestite nel modulo `rawmouse.hh`. In particolare, per limitazioni implementative di SFML, la funzione `sf::Event::MouseMovedRaw` non funziona in MacOS. Il metodo di riferimento è quello usato per i sistemi Windows e Linux; la variante per MacOS è solo una patch per ovviare a queste limitazioni. Questa patch potrebbe non essere necessaria con le release successive di SFML.

Estensioni proposte

Il codice proposto fin qui è estremamente rudimentale e poco generale. È pensato per mostrare un utilizzo di base delle tecniche fondamentali di programmazione grafica tridimensionale. Vi lasciamo qualche suggerimento che vada nella direzione di cominciare a generalizzarlo. Se implementaste tutti i suggerimenti, si avrebbe un'implementazione (ancora molto rudimentale e basilare) di una parte del cuore di un piccolo *game engine* (i game engine possono fare molte cose ed essere, a seconda dei loro casi d'uso, a livelli di astrazione molto diversi fra loro).

Usare shader diversi per oggetti diversi

Al momento, il programma usa un solo shader alla volta per tutta la scena. Può cambiare shader al volo, ma questo influenza tutto quanto. In realtà, oggetti diversi potrebbero richiedere shader diversi. Come minimo, vorremmo poter usare lo shader flat per oggetti come cubi e parallelepidi o in generale per oggetti a spigoli vivi, e lo shader phong per le sfere o in generale per oggetti tondeggianti. Per far ciò, bisogna estendere il modulo shader per poter tenere più programmi shader compilati ed attivi nello stesso momento e per attivarne l'uso di uno specifico prima disegnare una primitiva OpenGL, e di un altro prima di un'altra, all'interno della preparazione di un singolo frame, tramite `glUseProgram()`. Di per sé questo non sarebbe difficile, ma bisogna ricordare che cambiare uno shader vuol dire aggiornare tutti i valori location a cui scrivere le uniform di cui ha

bisogno! Questa gestione diventa per forza più complessa. Bisogna portare la gestione delle uniform dentro la classe che gestisce gli shader, e usare una struttura dati che mappi il nome della uniform a cui voglio scrivere al valore di location per lo shader attivo in quel momento. Il suggerimento più semplice è usare una struttura dati di tipo **map** come quelle fornite dalla STL del C++.

Grafo di scena

Costruire un grafo di scena che rappresenta una scena gerarchica simile a quella dei Capitoli 4 e 5. Questo esempio contiene già tre livelli di gerarchia: la stanza, i tavoli che vengono posizionati rispetto alla stanza e gli oggetti sui tavoli che sono posizionati rispetto ai tavoli stessi; inoltre, i parallelepipedi che compongono un tavolo vengono assemblati rispetto al tavolo stesso.

Al momento, questa gerarchia è creata esplicitamente nel codice tramite funzioni esplicite, che si richiamano fra di loro ed ereditano la trasformazione di modellazione corrente come parametro (matrice **parent_mm**). Ogni funzione è pensata per disegnare una sola cosa specifica, con dentro variabili locali tarate ad hoc.

In generale, nel grafo di scena, ogni oggetto figlio deve subire una trasformazione di modellazione che è la composizione della trasformazione che lo posiziona rispetto al padre con quella che posiziona il padre nel mondo. Invece di avere funzioni ad hoc, la cosa migliore è avere una struttura dati che contenga tutte le trasformazioni che ho bisogno di fare e tutti gli oggetti che posso voler disegnare, memorizzate in modo che la topologia della struttura dati corrisponda alle sequenze di trasformazioni necessarie per raggiungere il risultato desiderato.

La struttura dati più naturale a questo scopo è un albero, i cui nodi sono istanze degli oggetti disponibili, mentre gli archi codificano le trasformazioni che li mettono in relazione. L'albero deve essere attraversato, accumulando le matrici di modellazione nel modo opportuno, per visualizzare tutti gli oggetti della scena. La funzione disegno diventa quindi una visita completa dell'albero. Le informazioni specifiche per modellare ogni oggetto (semplice o complesso che sia) sono tutte dentro ai nodi e agli archi dell'albero, e non dentro al codice.

Luci e materiali multipli

Al momento, la classe **Lights** crea una sola luce e forza tutti gli oggetti disegnati nella scena ad avere un solo materiale. In realtà, la classe è solo un placeholder per le uniform locations da tenere pronte per essere inviate agli shader. Se è stata implementata l'estensione per poter usare shader diversi all'interno dello stesso frame / scena, la classe diventa inutile.

Per quanto riguarda i materiali, dovrebbe potercene essere uno diverso per ogni oggetto: il posto giusto per memorizzare il valore dei materiali sarebbe nei nodi del grafo di scena.

Per le luci, il discorso è complesso, e può diventare complicato a piacere (luci posizionali, luci direzionali, luci con cutoff, etc). Come minimo, la gestione

di un numero fisso di luci (o di un numero arbitrario e dinamico di luci) deve essere gestita e supportata negli shader che computano l'illuminazione. Se le luci possono essere un numero arbitrario, allora i dati delle luci (posizione, valori per componente etc) devono essere memorizzati in un buffer, che va trasferito alla GPU, e gli shader di illuminazione devono eseguire uno o più loop su quel buffer.